

84. Le décor qui défile

Un monde plus large que l'écran.

1. Ce qu'il faut comprendre

La carte de fond fait **32 cases sur 32**, soit 256 pixels de côté ; l'écran n'en montre que 20 sur 18. Le reste attend hors-champ.

`defiler(x, y)` fait glisser la fenêtre. Au-delà de 255, le compte revient tout seul à zéro : le décor est un **ruban sans fin**.

C'est ce qui permet un niveau aussi long qu'on veut : quand la caméra franchit une case, on redessine la colonne qui va entrer par la droite, à la place de celle qui vient de sortir par la gauche.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **595 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         99 o ← démarrage — ta fonction
VBlank         8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
AdresseCase    10 o ← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
LireManette    37 o ← bouton() — lit les huit boutons
Tuiles         208 o ← CopierTuiles — 12 lettres sur 43, et 1 tuile dessinée
"DROITE POUR AVA..." 19 o ← texte() — un texte gravé
```

Lettres gravées : ACDEINOPRTUV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()   × 2   26 o sur place, dans main() + AdresseCase (10 o)
texte()   × 1   11 o sur place, dans main() + EcrireTexte (10 o)
bouton()  × 2   10 o sur place, dans main() + LireManette (37 o)
```

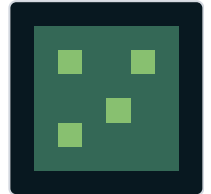
```
defiler() × 1      7 o sur place, dans main()
image()   × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>      // un dessin de 8 × 8 pixels
#include <texte>       // écrit un texte à l'écran
#include <poser>       // pose une tuile sur une case du fond
#include <bouton>      // lit un bouton de la manette
#include <defiler>     // fait glisser tout le fond
```

2. UN DESSIN — MUR

```
Tuile MUR = {
  "33333333",
  "32222223",
  "32122123",
  "32222223",
  "32221223",
  "32122223",
  "32222223",
  "33333333",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  uint8_t d = 0;

  texte(2, 2, "DROITE POUR AVANCER");

  for (uint8_t x = 0; x < 32; x++) {
    poser(x, 14, MUR);
    poser(x, 15, MUR);
  }

  while (true) {
    image();

    if (bouton(DROITE)) d++;
    if (bouton(GAUCHE)) d--;

    defiler(d, 0);
  }

  return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



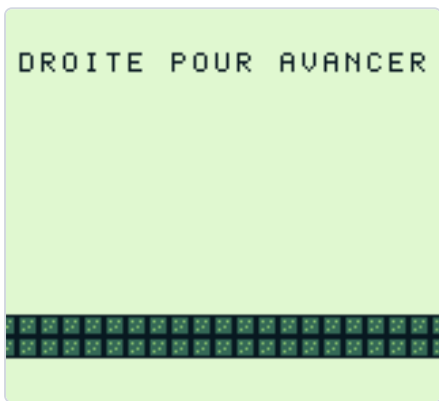
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Le sol glisse sous l'écran quand on pousse la croix.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le sol est dessiné
- ✓ sur toute la carte, hors-champ compris
- ✓ DROITE fait défiler (0 vers 20)

Ensuite : **85. La caméra : c'est le monde qui bouge** — chapitre 13 : Ce qui va ensemble. · gameboy3 — écrire une cartouche Game Boy en C++